MINIMAL PYTHON PARALLEL SYSTEMS LAB | UNGRADED PRACTICE SHEET

Performance Evaluation and Optimization in Parallel Computing

Scaling Laws, Shared-Memory Cache Contention, Empirical Roofline, and Dynamic Load Balancing

Prepared by: Sufyan bin Uzayr
Execution: Local Laptop (Offline, Pure Python 3)
Assessment: Ungraded Solo Exercise
Rules: No Paper, No Internet, No AI Tools

EXECUTION RULES AND HARDWARE-BOUND VERIFICATION

- **Offline, Pure Python 3 Only:** Disconnect Wi-Fi/Ethernet. Use only standard Python 3 (`multiprocessing`, `shared_memory`, `ctypes`, `time`, `math`). No scratch paper, external packages, or AI tools.
- **True Multi-Core Execution (Bypassing the GIL):** All parallel tasks must use OS processes (`mp.Process` / `mp.Pool`) and `SharedMemory` so workers physically exercise multi-core caches and memory controllers. Define all worker functions at module top level inside an `if __name__ == "__main__":` guard.
- **Chained Hardware Telemetry:** Each phase emits a 64-bit FNV-1a signature (`VERIFY_HASH`) bound to your laptop's logical core count and measured timings. Feed measured parameters forward across phases.

1. Phase 0: Hardware Topology & Minimal Telemetry Harness

Inspect your laptop's CPU and cache hierarchy via terminal and record the values in `lab_report_<YourID>.md`:

```
# Linux:   lscpu | grep -E "Model name|Core|Thread|L1d|L2|L3" && getconf LEVEL1_DCACHE_LINESIZE
# macOS:   sysctl -n machdep.cpu.brand_string && sysctl hw.physicalcpu hw.logicalcpu hw.
    cachelinesize hw.l1dcachesize hw.l2cachesize hw.l3cachesize
# Windows: Get-CimInstance Win32_Processor | Select Name, NumberOfCores, NumberOfLogicalProcessors
    , L2CacheSize, L3CacheSize
```

Record: **CPU Model**, **Physical Cores** (P_{phys}), **Logical Threads** ($P_{\log}$), **Cache Line** (L_{bytes}), and **L1d / L2 / L3 Sizes**. Next, save `harness.py`:

Listing 1: harness.py — Minimal Timing and Verification Harness

```python
from dataclasses import dataclass
import multiprocessing as mp
import time
from typing import Callable

@dataclass
class BenchStats:
    min_ms: float
    median_ms: float
    iqr_ms: float

def fnv1a_hash(s: str) -> int:
    h = 14695981039346656037
    for b in s.encode("utf-8"):
        h = ((h ^ b) * 1099511628211) & 0xFFFFFFFFFFFFFFFF
    return h

def run_benchmark(fn: Callable[[], None], warmup: int = 2, runs: int = 7) -> BenchStats:
    for _ in range(warmup):
        fn()
    samples = []
    for _ in range(runs):
        t0 = time.perf_counter_ns()
        fn()
        samples.append((time.perf_counter_ns() - t0) / 1e6)
    samples.sort()
    med = samples[runs // 2]
    return BenchStats(samples[0], med, samples[(3 * runs) // 4] - samples[runs // 4])

def emit_verify_tag(phase: str, seed: int, m1: float, m2: float) -> str:
    hw = mp.cpu_count()
    sig = fnv1a_hash(f"{phase}|{seed}|{hw}|{m1:.4f}|{m2:.4f}")
    return f"VERIFY_HASH[{phase}]: {sig:016X} (P_log={hw})"
```

Checkpoint (`task0_cal.py`): Query `mp.cpu_count()`, measure average nanosecond call latency over 10^6 consecutive

`time.perf_counter_ns()` calls, and emit `VERIFY_HASH[PHASE0]` with your Student ID as `seed`.

2. Phase 1: False Sharing Stride Sweep & Karp-Flatt Diagnostics

Allocate a 4096-byte `SharedMemory` block and spawn p worker processes (`mp.Process`) to execute $M = 1.6 \times 10^7$ total read-modify-write steps (M/p per worker) on 64-bit unsigned integer slots spaced by `stride`:

Listing 2: Core Worker for task1_falsesharing.py

```python
def fs_worker(shm_name: str, slot_idx: int, iters: int) -> None:
    shm = shared_memory.SharedMemory(name=shm_name)
    arr = (ctypes.c_uint64 * 512).from_buffer(shm.buf)
    acc = arr[slot_idx]
    for i in range(iters):
        acc = (acc * 6364136223846793005 + (i | 1)) & 0xFFFFFFFFFFFFFFFF
        arr[slot_idx] = acc  # Forces store to shared L1 cache line
    del arr
    shm.close()
```

1. **Stride Sweep ($p = P_{\text{phys}}$):** Measure median ms, IQR, and throughput (Mops/s) across `stride` $\in \{1, 2, 4, 8, 16, 32\}$ (8 to 256 bytes). Identify the byte offset where coherence contention drops sharply.
2. **Process Scaling ($p \in \{1, 2, 4, \ldots, P_{\log}\}$):** Run sweeps for `stride = 1` and `stride = 16`. Compute Speedup $S_p = T_1/T_p$, Efficiency $E_p = S_p/p$, and the Karp-Flatt experimental serial fraction ($p \geq 2$):

$$e(p) = \frac{\frac{1}{S_p} - \frac{1}{p}}{1 - \frac{1}{p}}$$

3. **Analysis & Tag:** Contrast how $e(p)$ evolves with p for `stride = 1` vs. `stride = 16`. Emit `VERIFY_HASH[PHASE1]` passing `median_ms(stride=1)` and `median_ms(stride=16)` at $p = P_{\text{phys}}$.

3. Phase 2: Strong Scaling vs. Weak Scaling (`task2_scaling.py`)

Build a two-stage pipeline per trial: **Stage A** runs serially in the parent process over the first 5% of elements ($0.05N$, 10 `math.sin`/`math.sqrt` ops/element); **Stage B** partitions all N elements across a persistent `mp.Pool(processes=p)` created outside the timed loop (30 FMA/trig ops/element, zero shared writes).

1. **Strong Scaling:** Fix $N = 1.2 \times 10^6$. Sweep every integer $p \in \{1, 2, \ldots, P_{\log}\}$. Record T_p^{strong}, S_p^{strong}, and E_p^{strong}. Compare observed $S_{P_{\log}}^{\text{strong}}$ against Amdahl's cap $S_\infty = 1/s$, where $s = T_{\text{Stage A}}(1)/T_1^{\text{strong}}$.
2. **Weak Scaling:** Set per-worker load $N_0 = 2.0 \times 10^5$ and total load $N(p) = p \cdot N_0$ for $p \in \{1, 2, \ldots, P_{\log}\}$. Record T_p^{weak}, Weak Efficiency $E_p^{\text{weak}} = T_1^{\text{weak}}/T_p^{\text{weak}}$, and Gustafson Scaled Speedup:

$$S_p^{\text{Gustafson}} = p - \alpha(p-1), \quad \alpha = \frac{T_{\text{Stage A}}(p)}{T_p^{\text{weak}}}$$

3. **Architectural Inflection & Tag:** Quantify the drop in marginal speedup across $p = P_{\text{phys}} \to P_{\text{phys}} + 1$ (SMT / P-core to E-core boundary). Emit `VERIFY_HASH[PHASE2]` using $S_{P_{\log}}^{\text{strong}}$ and $S_{P_{\log}}^{\text{Gustafson}}$.

4. Phase 3: Empirical Roofline & Cache-Blocked GEMM (`task3_roofline_gemm.py`)

4.1 Part 3A: Laptop Roofline Ceilings ($p = P_{\text{phys}}$)

- **DRAM Bandwidth ($\text{BW}_{\text{sustained}}$):** Allocate two 64 MB `SharedMemory` byte buffers (128 MB total). Time parallel `memoryview` slice copies (`mv_a[s:e] = mv_b[s:e]`) across P_{phys} workers: $\text{BW}_{\text{sustained}} = \frac{2 \times 64 \times 10^6}{T_{\text{median}} \times 10^9}$ GB/s.
- **Compute Ceiling ($P_{\text{sustained}}$):** Time an unrolled 16-variable scalar FMA loop performing 6.0×10^7 FLOPs per worker across P_{phys} processes to obtain $P_{\text{sustained}}$ (GFLOP/s) and Ridge Point $I^* = P_{\text{sustained}}/\text{BW}_{\text{sustained}}$.

4.2 Part 3B: Progressive GEMM & Tile Sweep ($N = 256$, $p = P_{\text{phys}}$)

Implement double-precision $C = A \times B$ on flat 1D buffers ($3 \times 256^2 \times 8$ B = 1.57 MB) across P_{phys} workers:

- **Variant 1 (Naive i-j-k):** Row-partitioned i, middle j, strided inner k over $B[k \cdot N + j]$.
- **Variant 2 (Reordered i-k-j):** Row-partitioned i, middle k hoisting `a_ik = A[i * N + k]`, unit-stride inner j.

- **Variant 3 (6-Loop Cache-Blocked):** Tile (i, j, k) by $BS \times BS$ with (i, k, j) micro-kernel order and scalar `a_ik` hoisting. Verify $\max |C_{ij}^{(3)} - C_{ij}^{(2)}| < 10^{-9}$ and sweep $BS \in \{8, 16, 32, 64, 96, 128\}$.

Compute GFLOP/s $= \frac{2N^3}{T_{\text{median}} \times 10^9}$, tile footprint $W_{\text{tile}}(BS) = 3 \times BS^2 \times 8$ B, and operational intensity $I_{\text{tiled}}(BS^*) \approx BS^*/12$. Compare BS^* against your L1d/L2 cache sizes, map Variants 1–3 onto your Roofline, and emit `VERIFY_HASH[PHASE3]` using GFLOP/s$_{\text{Variant1}}$ and GFLOP/s$_{\text{Variant3}}(BS^*)$.

5. Phase 4: Skewed Workloads & Dynamic Chunk Queue (`task4_scheduling.py`)

Execute a triangular skewed workload over $N = 8,000$ items across $p = P_{\text{phys}}$ workers, where item $i \in \{0, \ldots, N-1\}$ runs $K(i) = 1 + \lfloor (i/40)^2 \rfloor$ heavy trigonometric iterations (`sin`, `cos`, `sqrt`):

1. **Three Scheduling Strategies:** (1) **Contiguous Block Static**; (2) **Cyclic Round-Robin Static**; and (3) **Shared-Memory Dynamic Queue** using a synchronized index counter (`mp.Value('i', 0)`) across chunk sizes $C \in \{1, 8, 64, 512\}$.
2. **Imbalance Telemetry:** Record each worker's active runtime $t_{\text{worker}}[t]$ in a shared `mp.Array('d', p)` and compute total wall time T_{wall} plus the **Load Imbalance Factor**:

$$\lambda = \frac{\max_{0 \le t < p} t_{\text{worker}}[t]}{\frac{1}{p} \sum_{t=0}^{p-1} t_{\text{worker}}[t]}$$

3. **Analysis & Tag:** Explain the trade-off between $C = 1$ (synchronization contention) and $C = 512$ (tail imbalance) to justify your optimal C^*. Emit `VERIFY_HASH[PHASE4]` using T_{wall}(Block Static) and T_{wall}(Dynamic, C^*).

UNGRADED DIGITAL REPORT SUMMARY

Compile a single Markdown report (`lab_report_<YourID>.md`) containing: **(1)** Phase 0 hardware table and all five verbatim `VERIFY_HASH[PHASE0..PHASE4]` lines; **(2)** summary tables of your measured timings, speedups, Karp-Flatt $e(p)$, Roofline GFLOP/s, and λ imbalance factors; **(3)** concise hardware-grounded answers to each phase's diagnostic questions; and **(4)** your complete Python 3 scripts.